

A Generic Types and Structural Constraints

Though type-agnostic by default, the system accommodates ecosystems with parametric polymorphism and structural typing. This section outlines optional extensions for expressing type-level abstraction and constraints across modules, layered atop the Merkle-based model without modifying its core semantics.

A.1 Generic Type Aliases

Modules may define parameterized type aliases of the form:

$$\text{type } T\langle X_1, \dots, X_n \rangle = \tau$$

where τ may reference type parameters X_i . When imported, these aliases are instantiated with concrete types:

$$T\langle \sigma_1, \dots, \sigma_n \rangle$$

Instantiations are resolved into fully derived types within the importing module.

A.2 Structural Constraints

Type parameters may be constrained by structural predicates:

$$\text{type } \text{Map}\langle K : \text{Hashable}, V \rangle = \dots$$

Here, `Hashable` denotes a required property or interface, transitively resolved through the module graph. Finalization ensures each constraint is satisfied by evidence such as declared instances, structural matches, or subtype edges.

A.3 Finalization Semantics

Generic types are instantiated and expanded during finalization. Constraint satisfaction is resolved transitively via module inheritance, yielding deterministic, content-addressed types. The result must be fully resolved and non-parametric in the final Merkle tree.

A.4 Applications

These mechanisms enable:

- Parameterized types (`List<T>`, `Map<K, V>`).
- Generic algorithms over abstract constraints.
- Interface-based modularity without coupling to implementations.

Use of generics is optional. Ecosystems lacking type-level abstraction can ignore this layer without affecting correctness or resolution semantics.

B Connection to Existing Module Calculi

This appendix generalizes conventional module calculi by embedding deterministic, content-addressable resolution into the module structure itself. It draws conceptual links to higher-order module systems such as those in ML [2], as well as the mixin calculus [1], while extending them for decentralized environments.

Unlike traditional systems, this approach enforces consistent inheritance order via Merkle tree construction and cryptographic tracing, providing explicit override histories and resolution paths even in complex graphs.

Future Work. We anticipate extending this system with richer abstractions such as recursive types, function modules, and higher-order modules. These additions would support more expressive module ecosystems without compromising determinism or verifiability.

C Benchmarks and Evaluation Methodology

This appendix provides full details on the synthetic benchmark methodology, test scenarios, and comparative metrics used to evaluate the Merkle tree-based module system discussed in the main paper.

C.1 Synthetic Evaluation

To validate the model’s feasibility, we constructed synthetic benchmarks over simulated module graphs with controlled topologies:

- **Linear chains:** Deep ancestry resolution.
- **Wide graphs:** Shared base modules for deduplication tests.

- **Diamond graphs:** Shadowing and conflict resolution.

Each module contains N symbols and inherits from k parents, with $N \in [10, 1000]$ and $k \in [1, 10]$. Benchmarked operations include:

- **Finalization time:** Merkle root and inheritance trace computation.
- **Symbol resolution:** Lookup and latency analysis.
- **Graph compression:** Deduplication via structural sharing.

Disclaimer. These tests were conducted using a simulated interpreter, not a full runtime. Results are indicative of theoretical scalability, not real-world performance.

C.2 Results Summary

- Finalization scales linearly with symbol count and ancestry depth.
- Symbol lookup remains sublinear in common cases due to caching and reuse.
- Structural sharing yields 60–80% graph compression in shared-ancestor cases.

C.3 Prototype Implementation Plan

A prototype is planned in Rust, using:

- `libp2p` for decentralized lookup and DHT-based resolution.
- `Wasmer` to evaluate WebAssembly-based modules and execute finalization logic.

Tangl is one such prototype implementation of this model and will be used to empirically validate performance in future work.

C.4 Planned Evaluation Categories

- **Scalability:** Nested resolution, wide fan-out.
- **Fault tolerance:** Stale caches, resolution under failure.
- **Security:** Malicious ancestry, forked resolution graphs.
- **Tooling usability:** CLI/IDE feedback, trace inspection.
- **Workflow validation:** CI rebuilds, development diffs.

C.5 Comparison to Existing Systems

Tangl will be benchmarked against `npm`, `Cargo`, and `pip` on tasks including:

- Cold vs. warm start resolution.
- Incremental rebuilds.
- Registry fetch latency.

Metrics will include:

- DHT hops per module.
- Cache hit/miss ratios.
- Fallback success under partial connectivity.

D CI/CD and Version Control Integration

This appendix illustrates how the system can integrate into real-world workflows, using Tangl as an example implementation.

D.1 CI/CD Pipelines

Tangl enables efficient CI/CD by leveraging Merkle roots for incremental validation. Because finalization captures full ancestry, CI systems can:

- Cache Merkle roots of finalized subgraphs.
- Skip recomputation when inheritance trees are unchanged.
- Identify resolution-affecting changes early in the build pipeline.

D.2 Version Control and Tagging

Ancestry DAGs can be embedded in Git tags or commit annotations. This allows:

- Cryptographic validation of ancestry consistency.

- Detection of semantic changes from resolution graph diffs.
- Structural tagging schemes for symbolic reproducibility.

D.3 Tooling Extensions

Future versions of Tangl may support:

- IDE visualizations of symbol overrides.
- Git hooks for ancestry change detection.
- CI runners that enforce deterministic finalization traces.

These features position Tangl as a prototype that demonstrates the viability of secure, decentralized module resolution in modern software development workflows.

References

- [1] Giovanni Castagna. 1993. The Mixin Calculus: Its Theory and Applications. *Journal of Functional Programming* 3, 4 (1993), 517–568. doi:10.1017/S095679680000146X
- [2] John C. Mitchell. 1988. Modules and Abstract Types. In *Proceedings of the ACM SIGPLAN '88 Conference on Programming Language Design and Implementation (PLDI)*. ACM, New York, NY, 252–261. doi:10.1145/539133.539159